

Leraning Path Recommendation Using Reinforcement Learning

obs - The system generated a personalized learning path according to the student’s skill level, learning goal, learning style and daily study hours.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

Obs: The required Python libraries for data handling, numerical operations, visualization, and machine learning were imported successfully.

```
df = pd.read_csv("learning_path_dataset.csv")

print("Dataset loaded successfully!")
```

Dataset loaded successfully!

Obs: The learning path dataset was loaded successfully using Pandas.

```
df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 11 columns):
#   Column                Non-Null Count  Dtype  
---  -
0   student_id            1000 non-null   int64   
1   study_hours           1000 non-null   float64  
2   quiz_score            1000 non-null   float64  
3   completed_topics      1000 non-null   int64   
4   total_topics          1000 non-null   int64   
5   practice_time         1000 non-null   float64  
6   attendance            1000 non-null   float64  
7   skill_level           1000 non-null   object  
8   learning_goal         1000 non-null   object  
9   course_difficulty     1000 non-null   object  
10  recommended_topic     1000 non-null   object  
dtypes: float64(4), int64(3), object(4)
memory usage: 86.1+ KB
```

Obs: The dataset information shows the column names, data types, and number of non-null values in each column.

```
print("Shape:", df.shape)
```

Shape: (1000, 11)

Obs: The shape of the dataset shows the total number of rows and columns available for analysis.

```
print("First 5 records:")
display(df.head())
```

First 5 records:

	student_id	study_hours	quiz_score	completed_topics	total_topics	practice_time	attendance	skill_level	learning_goal	course_difficulty	recommended_topic
0	1	7.08	78.15	7	15	1.82	98.95	Advanced	Data Science	Hard	Advanced Concepts
1	2	1.40	70.99	13	15	4.01	65.41	Intermediate	Machine Learning	Medium	Advanced Concepts
2	3	4.09	99.48	11	15	3.88	94.72	Beginner	Web Development	Medium	Project Based Learning
3	4	6.80	83.24	0	15	0.42	53.52	Beginner	Machine	Hard	Advanced Concepts

Obs: The first five records were displayed to understand the structure and type of student learning information.

```
print("Last 5 records:")
display(df.tail())
```

Last 5 records:

	student_id	study_hours	quiz_score	completed_topics	total_topics	practice_time	attendance	skill_level	learning_goal	course_difficulty	recommended_topic
995	996	6.08	67.99	13	15	3.59	67.02	Beginner	Data Science	Hard	Intermediate Practice
996	997	4.27	75.17	1	15	6.87	60.54	Intermediate	Machine Learning	Easy	Advanced Concepts
997	998	5.16	47.76	8	15	2.19	77.81	Beginner	Data Science	Medium	Basics Revision
998	999	8.22	64.64	2	15	0.51	53.80	Beginner	Data Science	Medium	Intermediate

Obs: The last five records were displayed to check the end of the dataset and verify the available entries.

```
print("\nData types:")
print(df.dtypes)
```

```
Data types:
student_id      int64
study_hours     float64
quiz_score      float64
completed_topics int64
total_topics    int64
practice_time   float64
attendance      float64
skill_level     object
learning_goal   object
course_difficulty object
recommended_topic object
dtype: object
```

Obs: The data types of the dataset columns were checked to identify numerical and categorical features.

```
print("\nColumn names:")
print(df.columns.tolist())
```

```
Column names:
['student_id', 'study_hours', 'quiz_score', 'completed_topics', 'total_topics', 'practice_time', 'attendance', 'skill_level', 'learning_goal', 'course_difficulty', 'recommended_topic']
```

Obs: The column names were displayed to understand the different attributes used in the learning path recommendation system.

```
print("Statistical summary:")
display(df.describe())
```

```
Statistical summary:
```

	student_id	study_hours	quiz_score	completed_topics	total_topics	practice_time	attendance
count	1000.000000	1000.000000	1000.000000	1000.000000	1000.0	1000.000000	1000.000000
mean	500.500000	5.417220	64.083470	7.027000	15.0	4.146090	75.263570
std	288.819436	2.651007	20.133708	4.280724	0.0	2.350672	14.521838
min	1.000000	1.010000	30.020000	0.000000	15.0	0.000000	50.030000
25%	250.750000	3.140000	46.950000	3.000000	15.0	2.125000	62.722500
50%	500.500000	5.385000	63.060000	7.000000	15.0	4.150000	75.330000
75%	750.250000	7.742500	81.245000	11.000000	15.0	6.220000	88.117500
max	1000.000000	9.980000	99.990000	14.000000	15.0	8.000000	99.920000

Obs: The descriptive statistics provide information about the numerical learning features, including their mean, minimum, maximum, and quartile values.

```
print("\nUnique values:")
print(df.nunique())
```

```
Unique values:
student_id      1000
study_hours      600
quiz_score      935
completed_topics   15
total_topics      1
practice_time    574
attendance      912
skill_level       3
learning_goal     4
course_difficulty  3
recommended_topic  4
dtype: int64
```

Obs: The number of unique values in each column was checked to understand the variety of student information and learning categories.

```
print("Missing values:")
print(df.isnull().sum())
```

```
Missing values:
student_id      0
study_hours     0
quiz_score      0
completed_topics 0
total_topics    0
practice_time    0
attendance      0
skill_level     0
learning_goal    0
course_difficulty 0
recommended_topic 0
dtype: int64
```

Obs: The missing values in each column were checked to identify incomplete student records.

```
print("\nMissing value percentage:")
print((df.isnull().sum() / len(df) * 100).round(2))
```

```
Missing value percentage:
student_id      0.0
study_hours     0.0
quiz_score      0.0
```

```
completed_topics    0.0
total_topics        0.0
practice_time       0.0
attendance          0.0
skill_level         0.0
learning_goal       0.0
course_difficulty   0.0
recommended_topic   0.0
dtype: float64
```

Obs-The percentage of missing values in each column was calculated and displayed to identify columns requiring data cleaning.

```
print("Duplicate records:", df.duplicated().sum())

df = df.drop_duplicates().reset_index(drop=True)

print("Shape after duplicate removal:", df.shape)
```

```
Duplicate records: 0
Shape after duplicate removal: (1000, 11)
```

Obs: Duplicate records were removed from the dataset to avoid repeated entries and improve data quality.

```
numeric_cols = df.select_dtypes(
    include=np.number
).columns

categorical_cols = df.select_dtypes(
    exclude=np.number
).columns

for col in numeric_cols:
    df[col] = df[col].fillna(df[col].median())

for col in categorical_cols:
    df[col] = df[col].fillna(df[col].mode()[0])

print("Missing values after treatment:")
print(df.isnull().sum())
```

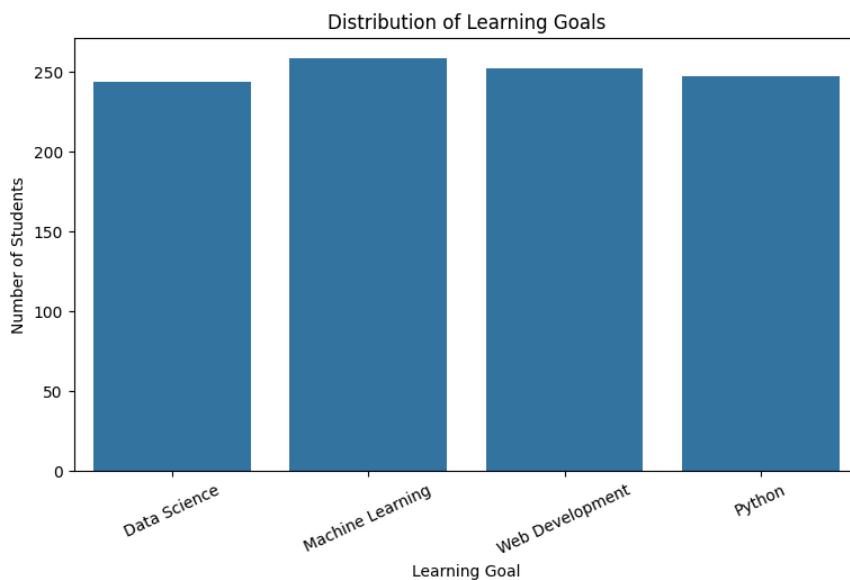
```
Missing values after treatment:
student_id      0
study_hours     0
quiz_score      0
completed_topics 0
total_topics    0
practice_time   0
attendance      0
skill_level     0
learning_goal   0
course_difficulty 0
recommended_topic 0
dtype: int64
```

Obs: Missing numerical values were replaced with the median, while missing categorical values were replaced with the most frequently occurring category

```
plt.figure(figsize=(9, 5))

sns.countplot(
    data=df,
    x="learning_goal"
)

plt.title("Distribution of Learning Goals")
plt.xlabel("Learning Goal")
plt.ylabel("Number of Students")
plt.xticks(rotation=25)
plt.show()
```

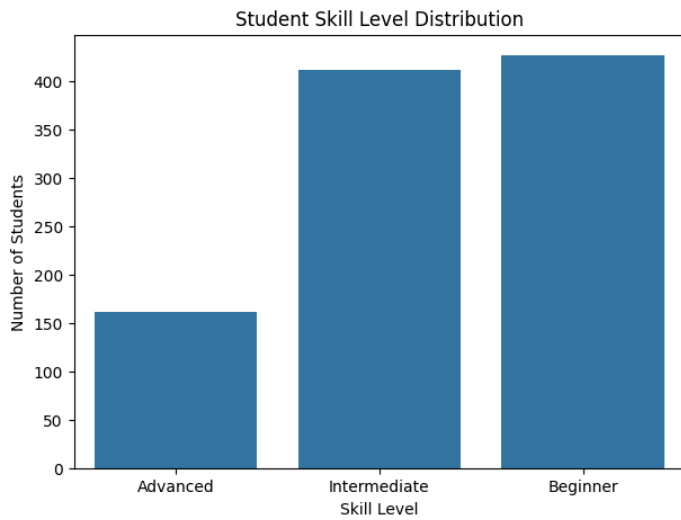


Obs: The countplot shows the distribution of different learning goals among students.

```
plt.figure(figsize=(7, 5))

sns.countplot(
    data=df,
    x="skill_level"
)

plt.title("Student Skill Level Distribution")
plt.xlabel("Skill Level")
plt.ylabel("Number of Students")
plt.show()
```

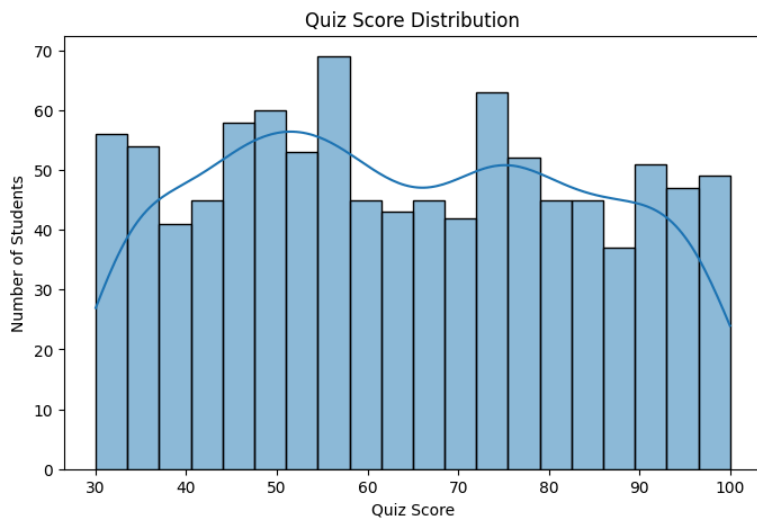


Obs: The graph shows the number of students belonging to different skill levels, such as beginner, intermediate, and advanced.

```
plt.figure(figsize=(8, 5))

sns.histplot(
    data=df,
    x="quiz_score",
    bins=20,
    kde=True
)

plt.title("Quiz Score Distribution")
plt.xlabel("Quiz Score")
plt.ylabel("Number of Students")
plt.show()
```

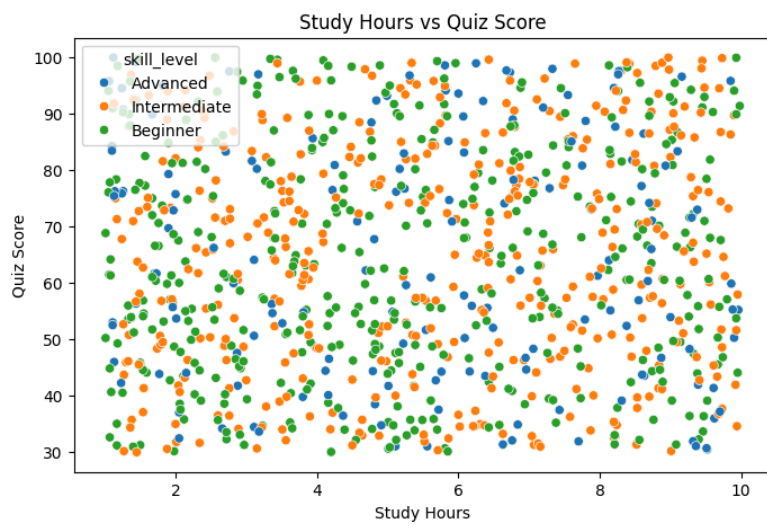


Obs: The histogram shows how quiz scores are distributed among students and helps understand their performance levels.

```
plt.figure(figsize=(8, 5))

sns.scatterplot(
    data=df,
    x="study_hours",
    y="quiz_score",
    hue="skill_level"
)

plt.title("Study Hours vs Quiz Score")
plt.xlabel("Study Hours")
plt.ylabel("Quiz Score")
plt.show()
```



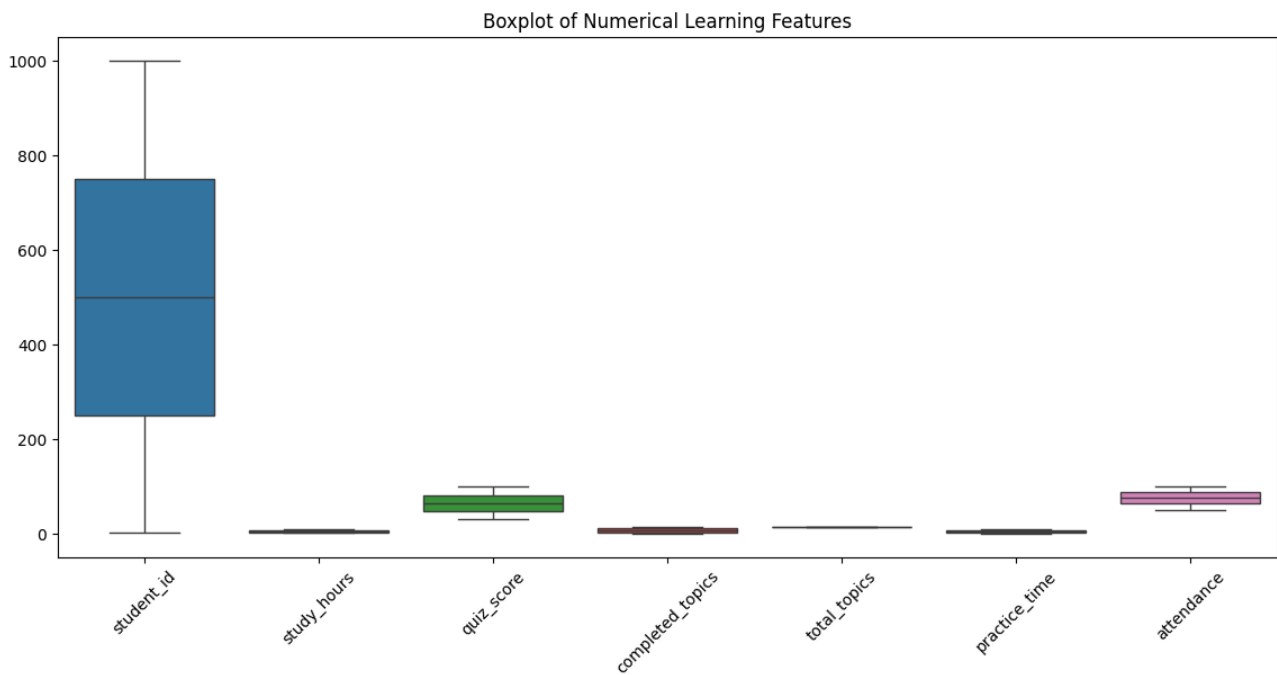
Obs: The scatterplot shows the relationship between study hours and quiz scores for students with different skill levels.

```
numeric_cols = df.select_dtypes(
    include=np.number
).columns.tolist()

plt.figure(figsize=(14, 6))

sns.boxplot(data=df[numeric_cols])

plt.title("Boxplot of Numerical Learning Features")
plt.xticks(rotation=45)
plt.show()
```



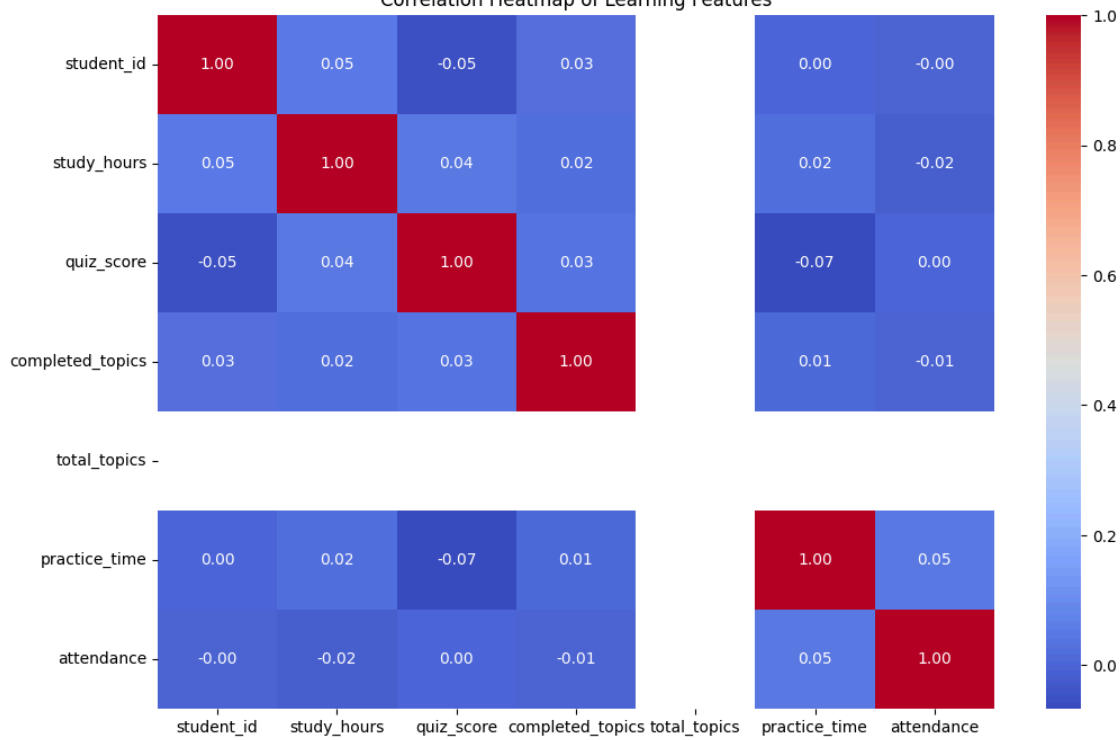
Obs: The boxplot displays the spread of numerical learning features and helps identify possible outliers.

```
plt.figure(figsize=(12, 8))

sns.heatmap(
    df[numeric_cols].corr(),
    annot=True,
    cmap="coolwarm",
    fmt=".2f"
)

plt.title("Correlation Heatmap of Learning Features")
plt.show()
```

Correlation Heatmap of Learning Features



Obs: The correlation heatmap shows the relationships between numerical learning features and helps identify related variables.

```
from scipy.stats import zscore

numeric_cols = df.select_dtypes(
    include=np.number
).columns.tolist()

z_scores = np.abs(
    zscore(df[numeric_cols])
)

outlier_count = (z_scores > 3).sum(axis=0)

outlier_df = pd.DataFrame({
    "Feature": numeric_cols,
    "Outlier Count": outlier_count
})

display(outlier_df)
```

/tmp/ipykernel_1615/554608358.py:8: RuntimeWarning: Precision loss occurred in moment calculation due to catastrophic cancellation. This occurs when the data are nearly constant.
zscore(df[numeric_cols])

	Feature	Outlier Count
0	student_id	0
1	study_hours	0
2	quiz_score	0
3	completed_topics	0
4	total_topics	0
5	practice_time	0
6	attendance	0

Next steps: [Generate code with outlier_df](#) [New interactive sheet](#)

Obs: The Z-score method was used to identify unusual values in numerical features that may affect the analysis.

```
df["progress_percentage"] = (
    df["completed_topics"] /
    df["total_topics"] * 100
)

df["learning_efficiency"] = (
    df["quiz_score"] /
    (df["study_hours"] + 1)
)

df["engagement_score"] = (
    df["study_hours"] +
    df["practice_time"] +
    df["attendance"] / 10
)

df["high_performer"] = (
    df["quiz_score"] >= 85
).astype(int)
```

```
print("Feature engineering completed!")
```

```
display(df.head())
```

Feature engineering completed!

	student_id	study_hours	quiz_score	completed_topics	total_topics	practice_time	attendance	skill_level	learning_goal	course_difficulty	recommended_topic	progress_percentage
0	1	7.08	78.15	7	15	1.82	98.95	Advanced	Data Science	Hard	Advanced Concepts	95.5
1	2	1.40	70.99	13	15	4.01	65.41	Intermediate	Machine Learning	Medium	Advanced Concepts	88.2
2	3	4.09	99.48	11	15	3.88	94.72	Beginner	Web Development	Medium	Project Based Learning	92.1
3	4	6.80	83.24	0	15	0.42	53.52	Beginner	Machine Learning	Hard	Advanced Concepts	78.9
4	5	3.56	50.93	7	15	4.45	71.30	Beginner	Python	Hard	Intermediate Practice	85.3

Obs: New features such as learning efficiency, engagement score, and high performer were created to represent student learning behaviour more effectively.

```
numeric_cols = df.select_dtypes(
    include=np.number
).columns.tolist()

skewness = df[numeric_cols].skew()

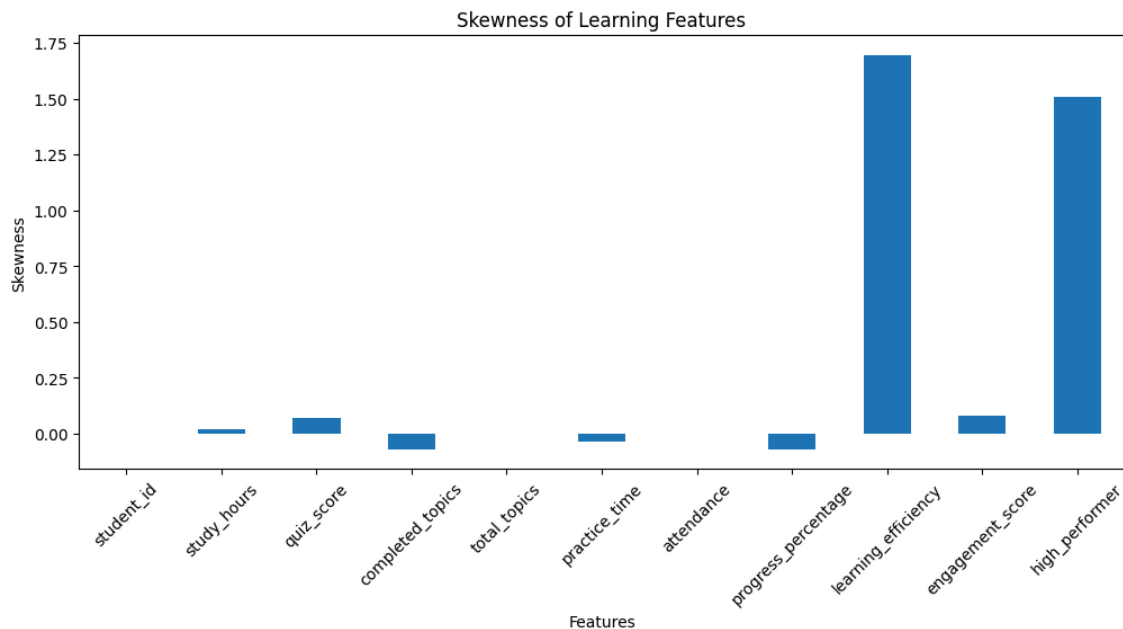
print("Skewness of numerical features:")
print(skewness.sort_values(ascending=False))

plt.figure(figsize=(12, 5))

skewness.plot(kind="bar")

plt.title("Skewness of Learning Features")
plt.xlabel("Features")
plt.ylabel("Skewness")
plt.xticks(rotation=45)
plt.show()
```

```
Skewness of numerical features:
learning_efficiency    1.695191
high_performer         1.510101
engagement_score       0.076405
quiz_score             0.069395
study_hours            0.017405
student_id             0.000000
total_topics           0.000000
attendance             -0.001074
practice_time          -0.037051
progress_percentage    -0.072242
completed_topics       -0.072242
dtype: float64
```



Obs: The skewness analysis shows the distribution of numerical features and helps identify features that are not normally distributed.

```
from sklearn.preprocessing import PowerTransformer, StandardScaler

numeric_cols = df.select_dtypes(
    include=np.number
).columns.tolist()

# Exclude student ID
numeric_cols = [
    col for col in numeric_cols
    if col != "student_id"
]
```

```

power = PowerTransformer(
    method="yeo-johnson"
)

transformed_data = power.fit_transform(
    df[numeric_cols]
)

transformed_df = pd.DataFrame(
    transformed_data,
    columns=numeric_cols
)

print("After PowerTransformer:")
display(transformed_df.head())

scaler = StandardScaler()

scaled_data = scaler.fit_transform(
    transformed_df
)

scaled_df = pd.DataFrame(
    scaled_data,
    columns=numeric_cols
)

print("After StandardScaler:")
display(scaled_df.head())

```

After PowerTransformer:

	study_hours	quiz_score	completed_topics	total_topics	practice_time	attendance	progress_percentage	learning_efficiency	engagement_score	high_performer	
0	0.656696	0.723226	0.063583	0.0	-0.971207	1.593400	0.100429	-0.093328	0.455791	-0.498437	
1	-1.613641	0.390698	1.322664	0.0	0.000835	-0.663678	1.278420	1.715415	-1.340251	-0.498437	
2	-0.441982	1.654016	0.919902	0.0	-0.053630	1.320754	0.907412	1.096152	0.112224	2.006272	
3	0.559601	0.952938	-1.796437	0.0	-1.688236	-1.529471	-1.926640	0.082975	-1.170528	-0.498437	
4	-0.654587	-0.613295	0.063583	0.0	0.182825	-0.249558	0.100429	0.163529	-0.484330	-0.498437	

After StandardScaler:

	study_hours	quiz_score	completed_topics	total_topics	practice_time	attendance	progress_percentage	learning_efficiency	engagement_score	high_performer	
0	0.656696	0.723226	0.063583	0.0	-0.971207	1.593400	0.100429	-0.093328	0.455791	-0.498437	
1	-1.613641	0.390698	1.322664	0.0	0.000835	-0.663678	1.278420	1.715415	-1.340251	-0.498437	
2	-0.441982	1.654016	0.919902	0.0	-0.053630	1.320754	0.907412	1.096152	0.112224	2.006272	
3	0.559601	0.952938	-1.796437	0.0	-1.688236	-1.529471	-1.926640	0.082975	-1.170528	-0.498437	
4	-0.654587	-0.613295	0.063583	0.0	0.182825	-0.249558	0.100429	0.163529	-0.484330	-0.498437	

Obs: Power transformation and standard scaling were applied to numerical features to improve their distribution and bring them to a common scale.

```

from sklearn.preprocessing import LabelEncoder

df_encoded = df.copy()

categorical_cols = df_encoded.select_dtypes(
    exclude=np.number
).columns.tolist()

encoders = {}

for col in categorical_cols:
    le = LabelEncoder()
    df_encoded[col] = le.fit_transform(
        df_encoded[col].astype(str)
    )
    encoders[col] = le

print("Label encoding completed!")

display(df_encoded.head())

```

Label encoding completed!

	student_id	study_hours	quiz_score	completed_topics	total_topics	practice_time	attendance	skill_level	learning_goal	course_difficulty	recommended_topic	pro
0	1	7.08	78.15	7	15	1.82	98.95	0	0	1	0	
1	2	1.40	70.99	13	15	4.01	65.41	2	1	2	0	
2	3	4.09	99.48	11	15	3.88	94.72	1	3	2	3	
3	4	6.80	83.24	0	15	0.42	53.52	1	1	1	0	
4	5	3.56	50.93	7	15	4.45	71.30	1	2	1	2	

Obs: Categorical features were converted into numerical values using LabelEncoder so that they could be used by machine learning models.

```

from sklearn.model_selection import train_test_split

X = df.drop(
    columns=["recommended_topic", "student_id"]
)

```



```

y = df["recommended_topic"]

label_encoder = LabelEncoder()

y_encoded = label_encoder.fit_transform(y)

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y_encoded,
    test_size=0.30,
    random_state=51,
    stratify=y_encoded
)

print("Training data:", X_train.shape)
print("Testing data:", X_test.shape)

```

```

Training data: (700, 13)
Testing data: (300, 13)

```

Obs: The dataset was divided into training and testing sets, with 70% used for training and 30% used for testing.

```

from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import (
    OneHotEncoder,
    StandardScaler
)

numeric_features = X_train.select_dtypes(
    include=np.number
).columns.tolist()

categorical_features = X_train.select_dtypes(
    exclude=np.number
).columns.tolist()

numeric_transformer = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler())
])

categorical_transformer = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("encoder", OneHotEncoder(
        handle_unknown="ignore",
        sparse_output=False
    ))
])

preprocessor = ColumnTransformer(
    transformers=[
        ("num", numeric_transformer, numeric_features),
        ("cat", categorical_transformer, categorical_features)
    ]
)

print("Preprocessing pipeline created!")

```

```

Preprocessing pipeline created!

```

Obs: A preprocessing pipeline was created to handle missing values, scale numerical features, and encode categorical features before model training.

```

from imblearn.over_sampling import SMOTE

# First transform training data
X_train_processed = preprocessor.fit_transform(X_train)

print("Before SMOTE:", X_train_processed.shape)

smote = SMOTE(random_state=51)

X_train_balanced, y_train_balanced = smote.fit_resample(
    X_train_processed,
    y_train
)

print("After SMOTE:", X_train_balanced.shape)

print("\nClass distribution after SMOTE:")
print(pd.Series(y_train_balanced).value_counts())

```

```

Before SMOTE: (700, 20)
After SMOTE: (828, 20)

Class distribution after SMOTE:
1    207
2    207
3    207
0    207
Name: count, dtype: int64

```

Obs: SMOTE was applied to the training data to balance the target classes and reduce the effect of class imbalance.

```

from sklearn.linear_model import LogisticRegression
from sklearn.naive_bayes import GaussianNB

```

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC

from sklearn.ensemble import (
    RandomForestClassifier,
    AdaBoostClassifier,
    GradientBoostingClassifier
)

from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score
)

models = {
    "Logistic Regression": LogisticRegression(max_iter=1000),
    "Gaussian Naive Bayes": GaussianNB(),
    "Decision Tree": DecisionTreeClassifier(random_state=51),
    "KNN": KNeighborsClassifier(),
    "SVM": SVC(probability=True, random_state=51),
    "Random Forest": RandomForestClassifier(
        n_estimators=100,
        random_state=51,
        n_jobs=-1
    ),
    "AdaBoost": AdaBoostClassifier(random_state=51),
    "Gradient Boosting": GradientBoostingClassifier(random_state=51)
}

results = []
trained_models = {}

for name, model in models.items():

    model.fit(X_train_balanced, y_train_balanced)

    pred = model.predict(
        preprocessor.transform(X_test)
    )

    results.append({
        "Model": name,
        "Accuracy": accuracy_score(y_test, pred),
        "Precision": precision_score(
            y_test, pred, average="weighted", zero_division=0
        ),
        "Recall": recall_score(
            y_test, pred, average="weighted", zero_division=0
        ),
        "F1 Score": f1_score(
            y_test, pred, average="weighted", zero_division=0
        )
    })

    trained_models[name] = model

results_df = pd.DataFrame(results)

display(results_df.sort_values(
    by="F1 Score",
    ascending=False
))
```

	Model	Accuracy	Precision	Recall	F1 Score	
2	Decision Tree	1.000000	1.000000	1.000000	1.000000	
5	Random Forest	1.000000	1.000000	1.000000	1.000000	
7	Gradient Boosting	1.000000	1.000000	1.000000	1.000000	
1	Gaussian Naive Bayes	0.980000	0.980134	0.980000	0.979993	
0	Logistic Regression	0.970000	0.970080	0.970000	0.969847	
4	SVM	0.953333	0.953479	0.953333	0.953256	
3	KNN	0.756667	0.757949	0.756667	0.753094	
6	AdaBoost	0.486667	0.302722	0.486667	0.351247	

Obs: Multiple classification models were trained on the processed data, and their accuracy, precision, recall, and F1-score were calculated for performance comparison.

```
from sklearn.metrics import (
    confusion_matrix,
    classification_report
)

best_name = results_df.sort_values(
    by="F1 Score",
    ascending=False
).iloc[0]["Model"]

best_model = trained_models[best_name]

pred = best_model.predict(
    preprocessor.transform(X_test)
)

print("Best Model:", best_name)
```

```

print("\nClassification Report:")
print(classification_report(
    y_test,
    pred,
    target_names=label_encoder.classes_,
    zero_division=0
))

cm = confusion_matrix(y_test, pred)

plt.figure(figsize=(7, 6))

sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    cmap="Blues",
    xticklabels=label_encoder.classes_,
    yticklabels=label_encoder.classes_
)

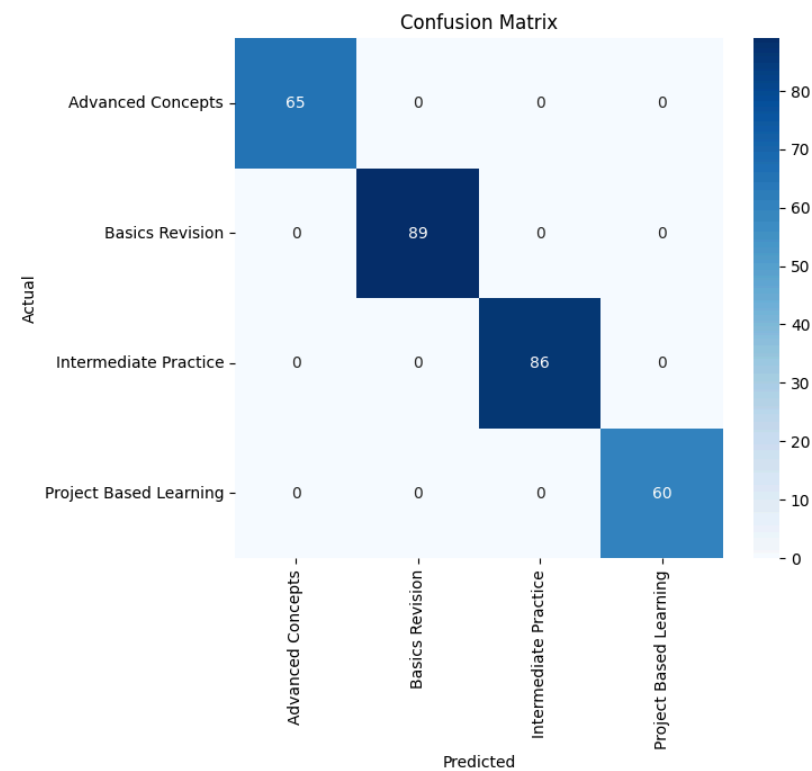
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")
plt.show()

```

Best Model: Decision Tree

Classification Report:

	precision	recall	f1-score	support
Advanced Concepts	1.00	1.00	1.00	65
Basics Revision	1.00	1.00	1.00	89
Intermediate Practice	1.00	1.00	1.00	86
Project Based Learning	1.00	1.00	1.00	60
accuracy			1.00	300
macro avg	1.00	1.00	1.00	300
weighted avg	1.00	1.00	1.00	300



Obs: The model with the highest weighted F1-score was selected as the best-performing model for further evaluation and prediction.

```

from sklearn.preprocessing import label_binarize
from sklearn.metrics import roc_auc_score, roc_curve

y_test_binary = label_binarize(
    y_test,
    classes=np.arange(len(label_encoder.classes_))
)

probabilities = best_model.predict_proba(
    preprocessor.transform(X_test)
)

roc_auc = roc_auc_score(
    y_test_binary,
    probabilities,
    multi_class="ovr",
    average="weighted"
)

print("Weighted ROC-AUC:", roc_auc)

plt.figure(figsize=(8, 6))

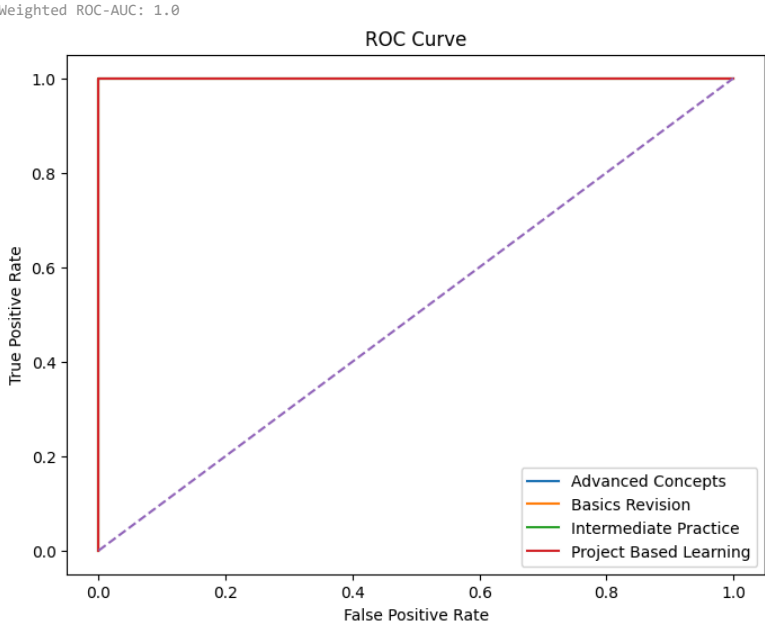
```

```
for i, class_name in enumerate(label_encoder.classes_):
    fpr, tpr, _ = roc_curve(
        y_test_binary[:, i],
        probabilities[:, i]
    )

    plt.plot(
        fpr,
        tpr,
        label=class_name
    )

plt.plot([0, 1], [0, 1], linestyle="--")

plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve")
plt.legend()
plt.show()
```



Obs: The confusion matrix and classification report were used to evaluate the correct and incorrect predictions made by the selected model.

```
from sklearn.model_selection import StratifiedKFold, cross_val_score

cv = StratifiedKFold(
    n_splits=5,
    shuffle=True,
    random_state=51
)

cv_results = []

for name, model in models.items():

    scores = cross_val_score(
        model,
        X_train_balanced,
        y_train_balanced,
        cv=cv,
        scoring="f1_weighted"
    )

    cv_results.append({
        "Model": name,
        "Mean CV F1": scores.mean(),
        "Std CV F1": scores.std()
    })

cv_df = pd.DataFrame(cv_results)

display(cv_df.sort_values(
    by="Mean CV F1",
    ascending=False
))
```

	Model	Mean CV F1	Std CV F1
2	Decision Tree	0.997590	0.002952
7	Gradient Boosting	0.997590	0.002952
5	Random Forest	0.997583	0.002960
0	Logistic Regression	0.969725	0.016774
1	Gaussian Naive Bayes	0.964750	0.017974
4	SVM	0.930280	0.018251
3	KNN	0.779965	0.009180
6	AdaBoost	0.370439	0.015211

Obs: The ROC curve and weighted ROC-AUC score were used to evaluate the model's classification performance across different recommendation classes.

```
from sklearn.model_selection import GridSearchCV

rf = RandomForestClassifier(
    random_state=51,
    n_jobs=-1
)

param_grid = {
    "n_estimators": [100, 200],
    "max_depth": [None, 10, 20],
    "min_samples_split": [2, 5],
    "min_samples_leaf": [1, 2]
}

grid_search = GridSearchCV(
    estimator=rf,
    param_grid=param_grid,
    cv=3,
    scoring="f1_weighted",
    n_jobs=-1,
    verbose=1
)

grid_search.fit(
    X_train_balanced,
    y_train_balanced
)

print("Best Parameters:")
print(grid_search.best_params_)

print("\nBest CV Score:")
print(grid_search.best_score_)

Fitting 3 folds for each of 24 candidates, totalling 72 fits
Best Parameters:
{'max_depth': None, 'min_samples_leaf': 1, 'min_samples_split': 5, 'n_estimators': 200}

Best CV Score:
0.9987922071102243
```

obs-GridSearchCV was used to find the best hyperparameters for the Random Forest model, and the best parameter combination along with its cross-validation score was displayed.

```
tuned_model = grid_search.best_estimator_

tuned_pred = tuned_model.predict(
    preprocessor.transform(X_test)
)

print("Tuned Random Forest Evaluation")

print("Accuracy:", accuracy_score(y_test, tuned_pred))

print("Precision:", precision_score(
    y_test, tuned_pred, average="weighted"
))

print("Recall:", recall_score(
    y_test, tuned_pred, average="weighted"
))

print("F1 Score:", f1_score(
    y_test, tuned_pred, average="weighted"
))

Tuned Random Forest Evaluation
Accuracy: 1.0
Precision: 1.0
Recall: 1.0
F1 Score: 1.0
```

Obs-The tuned Random Forest model was evaluated on the test data using accuracy, precision, recall, and weighted F1-score to measure its performance.

```
# Model Comparison

from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
import pandas as pd

models = {
    "Logistic Regression": LogisticRegression(max_iter=1000),
    "Decision Tree": DecisionTreeClassifier(random_state=42),
    "Random Forest": RandomForestClassifier(random_state=42),
    "Gradient Boosting": GradientBoostingClassifier(random_state=42)
}

results = []

for name, model in models.items():
    model.fit(
        preprocessor.transform(X_train),
```

```
        y_train

    )

    y_pred = model.predict(
        preprocessor.transform(X_test)
    )

    results.append({
        "Model": name,
        "Accuracy": accuracy_score(y_test, y_pred),
        "Precision": precision_score(
            y_test, y_pred, average="weighted", zero_division=0
        ),
        "Recall": recall_score(
            y_test, y_pred, average="weighted", zero_division=0
        ),
        "F1 Score": f1_score(
            y_test, y_pred, average="weighted", zero_division=0
        )
    })

results_df = pd.DataFrame(results)

display(results_df)
```

	Model	Accuracy	Precision	Recall	F1 Score
0	Logistic Regression	0.966667	0.966667	0.966667	0.966667
1	Decision Tree	1.000000	1.000000	1.000000	1.000000
2	Random Forest	0.996667	0.996705	0.996667	0.996663
3	Gradient Boosting	1.000000	1.000000	1.000000	1.000000

Next steps: [Generate code with results_df](#) [New interactive sheet](#)

Obs-Four machine learning models were trained and compared using accuracy, precision, recall, and weighted F1-score, and their performance results were displayed in a comparison table.

```
# Best Model Selection

best_model_name = results_df.loc[
    results_df["F1 Score"].idxmax(),
    "Model"
]

print("Best Model:", best_model_name)
```

Best Model: Decision Tree

Obs-The model with the highest weighted F1-score was selected as the best-performing model for learning path recommendation.

```
# Save Best Model

import joblib

best_model = models[best_model_name]

joblib.dump(best_model, "best_learning_path_model.pkl")

print("Best model saved successfully")
```

Best model saved successfully

Obs-The selected best-performing model was saved successfully as best_learning_path_model.pkl using the Joblib library for future predictions.

```
feature_names = preprocessor.get_feature_names_out()

importance_df = pd.DataFrame({
    "Feature": feature_names,
    "Importance": tuned_model.feature_importances_
})

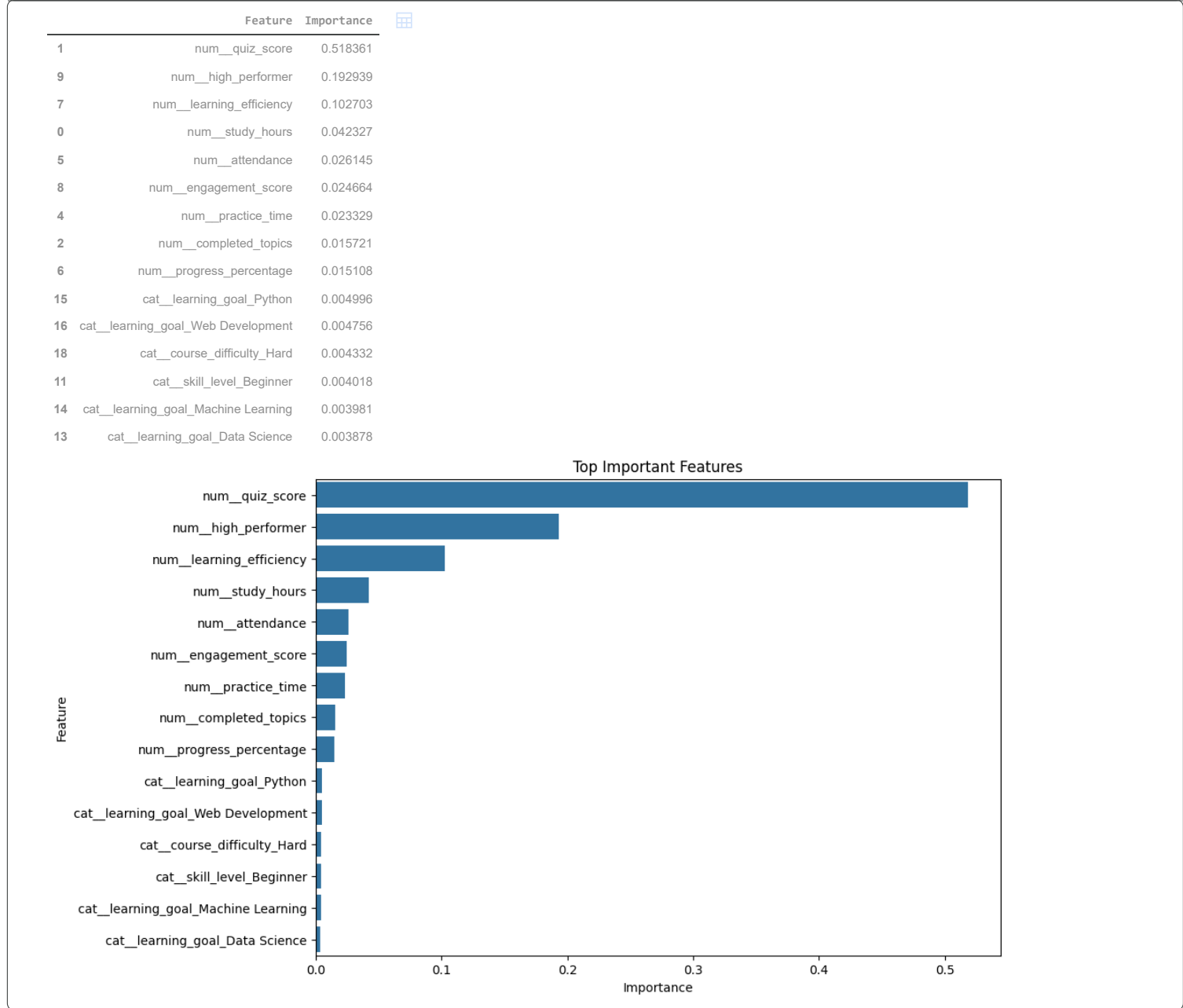
importance_df = importance_df.sort_values(
    by="Importance",
    ascending=False
)

display(importance_df.head(15))

plt.figure(figsize=(10, 7))

sns.barplot(
    data=importance_df.head(15),
    x="Importance",
    y="Feature"
)

plt.title("Top Important Features")
plt.show()
```



Obs-The importance of each feature was calculated using the tuned Random Forest model, and the top 15 important features were displayed through a sorted table and bar chart.

```
import random

states = [
    "Beginner_Low",
    "Beginner_Medium",
    "Intermediate_Low",
    "Intermediate_Medium",
    "Advanced_High"
]

actions = [
    "Python Basics",
    "Python Practice",
    "Data Analysis",
    "Machine Learning",
    "Project Based Learning"
]

q_table = pd.DataFrame(
    0.0,
    index=states,
    columns=actions
)

alpha = 0.1
gamma = 0.9
epsilon = 0.2
episodes = 1000

def get_state(skill, score):

    if skill == "Beginner" and score < 60:
        return "Beginner_Low"

    elif skill == "Beginner":
        return "Beginner_Medium"

    elif skill == "Intermediate" and score < 70:
```

```
        return "Intermediate_Low"

    elif skill == "Intermediate":
        return "Intermediate_Medium"

    else:
        return "Advanced_High"

def get_reward(action, score, progress):

    reward = 0

    if action == "Python Basics" and score < 60:
        reward += 10

    if action == "Python Practice" and 60 <= score < 75:
        reward += 10

    if action == "Data Analysis" and 70 <= score < 85:
        reward += 10

    if action == "Machine Learning" and score >= 75:
        reward += 10

    if action == "Project Based Learning" and score >= 85:
        reward += 15

    reward += progress * 0.1

    return reward

for episode in range(1000):

    row = df.sample(1).iloc[0]

    state = get_state(
        row["skill_level"],
        row["quiz_score"]
    )

    if random.random() < epsilon:
        action = random.choice(actions)
    else:
        action = q_table.loc[state].idxmax()

    reward = get_reward(
        action,
        row["quiz_score"],
        row["progress_percentage"]
    )

    next_state = state

    old_value = q_table.loc[state, action]

    next_max = q_table.loc[next_state].max()

    q_table.loc[state, action] = old_value + alpha * (
        reward + gamma * next_max - old_value
    )

print("Q-learning training completed!")

display(q_table)
```

Q-learning training completed!

	Python Basics	Python Practice	Data Analysis	Machine Learning	Project Based Learning
Beginner_Low	18.123130	13.261506	38.324327	20.140303	9.771738
Beginner_Medium	37.807026	25.088176	16.934971	19.276856	23.037956
Intermediate_Low	107.951489	33.441335	54.161907	34.211161	28.046813
Intermediate_Medium	40.366728	16.097336	22.664998	14.455490	10.247846
Advanced_High	12.620028	45.678836	19.943389	13.237079	7.774262

Next steps: [Generate code with q_table](#) [New interactive sheet](#)

Obs-The Q-learning model was trained for 1000 episodes using defined states, actions, rewards, and Q-value updates. The final Q-table was displayed to show the learned learning-topic recommendations.

```
def recommend_learning_path(skill, score):

    state = get_state(skill, score)

    recommendation = q_table.loc[state].idxmax()

    return state, recommendation

state, recommendation = recommend_learning_path(
    "Beginner",
    55
)

print("Student State:", state)
print("Recommended Learning Topic:", recommendation)
```

Student State: Beginner_Low
Recommended Learning Topic: Data Analysis

Obs-The Q-learning model used the student's skill level and score to determine the current state and recommend the most suitable learning topic.

Dashboard

```
import joblib

joblib.dump(
    preprocessor,
    "learning_preprocessor.pkl"
)

joblib.dump(
    tuned_model,
    "learning_path_model.pkl"
)

joblib.dump(
    label_encoder,
    "target_encoder.pkl"
)

joblib.dump(
    q_table,
    "q_learning_table.pkl"
)

print("All models saved successfully!")
```

All models saved successfully!

Obs: The preprocessing pipeline, trained model, target encoder, and Q-learning table were saved successfully for future use.

```
!pip install streamlit
```

Collecting streamlit
 Downloading streamlit-1.64.0-py3-none-any.whl.metadata (10 kB)
Requirement already satisfied: altair!=5.4.0,!5.4.1,<7,>=5.0.0 in /usr/local/lib/python3.13/dist-packages (from streamlit) (5.5.0)
Requirement already satisfied: click<9,>=7.0 in /usr/local/lib/python3.13/dist-packages (from streamlit) (8.5.0)
Requirement already satisfied: numpy<3,>=1.23 in /usr/local/lib/python3.13/dist-packages (from streamlit) (2.1.3)
Requirement already satisfied: packaging<20 in /usr/local/lib/python3.13/dist-packages (from streamlit) (26.3)
Requirement already satisfied: pandas<4,>=1.4.0 in /usr/local/lib/python3.13/dist-packages (from streamlit) (2.2.3)
Requirement already satisfied: pillow<13,>=7.1.0 in /usr/local/lib/python3.13/dist-packages (from streamlit) (11.3.0)
Collecting pydeck<1,>=0.8.0b4 (from streamlit)
 Downloading pydeck-0.9.3-py2.py3-none-any.whl.metadata (4.2 kB)
Requirement already satisfied: protobuf<8,>=5.26.1 in /usr/local/lib/python3.13/dist-packages (from streamlit) (5.29.6)
Requirement already satisfied: pyarrow!=25.0.0,<26,>=7.0 in /usr/local/lib/python3.13/dist-packages (from streamlit) (23.0.1)
Requirement already satisfied: requests<3,>=2.27 in /usr/local/lib/python3.13/dist-packages (from streamlit) (2.32.4)
Requirement already satisfied: tomli<2,>=0.10.1 in /usr/local/lib/python3.13/dist-packages (from streamlit) (0.10.2)
Requirement already satisfied: typing-extensions<5,>=4.10.0 in /usr/local/lib/python3.13/dist-packages (from streamlit) (4.16.0)
Requirement already satisfied: starlette<2,>=0.46.0 in /usr/local/lib/python3.13/dist-packages (from streamlit) (1.6.0)
Requirement already satisfied: uvicorn<1,>=0.30.0 in /usr/local/lib/python3.13/dist-packages (from streamlit) (0.52.4)
Requirement already satisfied: httpxtools<1,>=0.6.3 in /usr/local/lib/python3.13/dist-packages (from streamlit) (0.8.0)
Requirement already satisfied: anyio<5,>=4.0.0 in /usr/local/lib/python3.13/dist-packages (from streamlit) (4.14.2)
Requirement already satisfied: python-multipart<1,>=0.0.10 in /usr/local/lib/python3.13/dist-packages (from streamlit) (0.0.32)
Requirement already satisfied: websockets<17,>=12.0.0 in /usr/local/lib/python3.13/dist-packages (from streamlit) (15.0.1)
Requirement already satisfied: itsdangerous<3,>=2.1.2 in /usr/local/lib/python3.13/dist-packages (from streamlit) (2.2.0)
Requirement already satisfied: watchdog<7,>=2.1.5 in /usr/local/lib/python3.13/dist-packages (from streamlit) (6.0.0)
Requirement already satisfied: Jinja2 in /usr/local/lib/python3.13/dist-packages (from altair!=5.4.0,!5.4.1,<7,>=5.0.0->streamlit) (3.1.6)
Requirement already satisfied: jsonschema<3.0 in /usr/local/lib/python3.13/dist-packages (from altair!=5.4.0,!5.4.1,<7,>=5.0.0->streamlit) (4.26.0)
Requirement already satisfied: narwhals<1.14.2 in /usr/local/lib/python3.13/dist-packages (from altair!=5.4.0,!5.4.1,<7,>=5.0.0->streamlit) (2.25.0)
Requirement already satisfied: idna<=2.8 in /usr/local/lib/python3.13/dist-packages (from anyio<5,>=4.0.0->streamlit) (3.19)
Requirement already satisfied: python-dateutil<=2.8.2 in /usr/local/lib/python3.13/dist-packages (from pandas<4,>=1.4.0->streamlit) (2.9.0.post0)
Requirement already satisfied: pytz<=2020.1 in /usr/local/lib/python3.13/dist-packages (from pandas<4,>=1.4.0->streamlit) (2025.2)
Requirement already satisfied: tzdata<=2022.7 in /usr/local/lib/python3.13/dist-packages (from pandas<4,>=1.4.0->streamlit) (2026.3)
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.13/dist-packages (from requests<3,>=2.27->streamlit) (3.4.9)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.13/dist-packages (from requests<3,>=2.27->streamlit) (2.5.0)
Requirement already satisfied: certifi<=2017.4.17 in /usr/local/lib/python3.13/dist-packages (from requests<3,>=2.27->streamlit) (2026.7.22)
Requirement already satisfied: h11<=0.8 in /usr/local/lib/python3.13/dist-packages (from uvicorn<1,>=0.30.0->streamlit) (0.16.0)
Requirement already satisfied: MarkupSafe<=2.0 in /usr/local/lib/python3.13/dist-packages (from Jinja2->altair!=5.4.0,!5.4.1,<7,>=5.0.0->streamlit) (3.0.3)
Requirement already satisfied: attrs<=22.2.0 in /usr/local/lib/python3.13/dist-packages (from jsonschema<3.0->altair!=5.4.0,!5.4.1,<7,>=5.0.0->streamlit) (26.1.0)
Requirement already satisfied: jsonschema-specifications<=2023.03.6 in /usr/local/lib/python3.13/dist-packages (from jsonschema<3.0->altair!=5.4.0,!5.4.1,<7,>=5.0.0->streamlit) (2026.6.3)
Requirement already satisfied: referencing<=0.28.4 in /usr/local/lib/python3.13/dist-packages (from jsonschema<3.0->altair!=5.4.0,!5.4.1,<7,>=5.0.0->streamlit) (0.37)
Requirement already satisfied: rpds-py<=0.25.0 in /usr/local/lib/python3.13/dist-packages (from jsonschema<3.0->altair!=5.4.0,!5.4.1,<7,>=5.0.0->streamlit) (0.20.0)
Requirement already satisfied: six<=1.5 in /usr/local/lib/python3.13/dist-packages (from python-dateutil<=2.8.2->pandas<4,>=1.4.0->streamlit) (1.17.0)
Downloading streamlit-1.64.0-py3-none-any.whl (10.1 MB)
 10.1/10.1 MB 66.2 MB/s eta 0:00:00
Downloading pydeck-0.9.3-py2.py3-none-any.whl (11.4 MB)
 11.4/11.4 MB 69.2 MB/s eta 0:00:00
Installing collected packages: pydeck, streamlit
Successfully installed pydeck-0.9.3 streamlit-1.64.0

obs-The Flask server was created in app.py to load the saved learning path model and provide recommendations through the web application.

```
%writefile app.py
import streamlit as st
import pandas as pd
import numpy as np
from pathlib import Path
import random

# =====
# PAGE SETTINGS
# =====

st.set_page_config(
    page_title="LearnPath AI",
    page_icon="🧠",
    layout="wide",
    initial_sidebar_state="expanded"
)
```

```
# =====
# CUSTOM CSS
# =====

st.markdown("""
<style>
@import url('https://fonts.googleapis.com/css2?family=Inter:wght@400;500;600;700;800&display=swap');

* {
    font-family: 'Inter', sans-serif;
}

.stApp {
    background: #0b1020;
    color: #f8fafc;
}

[data-testid="stSidebar"] {
    background: #111827;
    border-right: 1px solid #273449;
}

[data-testid="stSidebar"] * {
    color: #e5e7eb !important;
}

.block-container {
    padding-top: 2rem;
    padding-bottom: 2rem;
    max-width: 1400px;
}

h1, h2, h3, h4, p, label {
    color: #f8fafc !important;
}

.main-heading {
    font-size: 38px;
    font-weight: 800;
    margin-bottom: 4px;
}

.sub-heading {
    color: #94a3b8 !important;
    font-size: 15px;
    margin-bottom: 25px;
}

.hero {
    background:
        radial-gradient(circle at 85% 15%, rgba(139, 92, 246, .45), transparent 30%),
        linear-gradient(135deg, #312e81, #4f46e5, #7c3aed);
    border-radius: 24px;
    padding: 34px;
    margin-bottom: 25px;
}

.hero-title {
    font-size: 30px;
    font-weight: 800;
    color: white !important;
}

.hero-text {
    color: #ddd6fe !important;
    font-size: 15px;
    line-height: 1.7;
}

.badge {
    display: inline-block;
    background: rgba(255,255,255,.15);
    color: white;
    padding: 7px 13px;
    border-radius: 30px;
    font-size: 12px;
    font-weight: 700;
    margin-bottom: 15px;
}

.metric-card,
.course-card,
.roadmap,
.info-panel {
    background: #151d31;
    border: 1px solid #273449;
    border-radius: 18px;
    padding: 22px;
}

.metric-card {
    min-height: 135px;
}

.metric-label {
    color: #94a3b8 !important;
    font-size: 12px;
    font-weight: 700;
    letter-spacing: .7px;
}

.metric-value {
    color: #ffffff !important;
    font-size: 25px;

```

```

font-weight: 800;
margin-top: 12px;
}

.section-title {
    color: white !important;
    font-size: 23px;
    font-weight: 800;
    margin-top: 32px;
    margin-bottom: 16px;
}

.course-card {
    min-height: 175px;
}

.course-icon {
    font-size: 30px;
    margin-bottom: 12px;
}

.course-title {
    color: white !important;
    font-size: 17px;
    font-weight: 700;
}

.course-description {
    color: #94a3b8 !important;
    font-size: 13px;
    line-height: 1.6;
    margin-top: 8px;
}

.step-number {
    color: #a78bfa !important;
    font-size: 12px;
    font-weight: 800;
    letter-spacing: 1px;
}

.step-title {
    color: white !important;
    font-size: 17px;
    font-weight: 700;
    margin-top: 6px;
}

.step-description {
    color: #94a3b8 !important;
    font-size: 13px;
    margin-top: 5px;
}

.info-panel h4 {
    color: #c4b5fd !important;
}

.info-panel p {
    color: #cbd5e1 !important;
    line-height: 1.7;
    font-size: 14px;
}

.stButton > button {
    background: linear-gradient(90deg, #6366f1, #8b5cf6);
    color: white;
    border: none;
    border-radius: 12px;
    font-weight: 700;
    padding: 12px;
}

.stButton > button:hover {
    background: linear-gradient(90deg, #818cf8, #a78bfa);
}

.footer {
    text-align: center;
    color: #64748b !important;
    font-size: 12px;
    margin-top: 45px;
}
</style>
""" , unsafe_allow_html=True)

# =====
# DATASET LOAD
# =====

DATASET_NAME = "learning_path_dataset.csv"
dataset_path = Path(DATASET_NAME)

dataset = None
dataset_error = None

if dataset_path.exists():
    try:
        dataset = pd.read_csv(dataset_path)
    except Exception as error:
        dataset_error = str(error)

# =====
# SESSION STATE
# =====

```

```

if "q_table" not in st.session_state:
    st.session_state.q_table = {}

if "recommended_path" not in st.session_state:
    st.session_state.recommended_path = []

if "progress" not in st.session_state:
    st.session_state.progress = 0

if "reward" not in st.session_state:
    st.session_state.reward = 0

# =====
# LEARNING CONTENT
# =====

LEARNING_PATHS = {
    "Python Programming": [
        ("Python Basics", "Learn syntax, variables and data types."),
        ("Conditions and Loops", "Understand decisions, loops and repetition."),
        ("Functions", "Create reusable and organized programs."),
        ("Object-Oriented Programming", "Learn classes, objects and inheritance."),
        ("Python Project", "Build a practical mini project.")
    ],
    "Data Science": [
        ("Python for Data Science", "Learn Python libraries for data analysis."),
        ("NumPy and Pandas", "Work with arrays, tables and datasets."),
        ("Data Cleaning", "Handle missing, duplicate and incorrect values."),
        ("Exploratory Data Analysis", "Discover patterns and relationships."),
        ("Data Visualization Project", "Create charts and present insights.")
    ],
    "Machine Learning": [
        ("Python and Mathematics", "Revise Python, statistics and mathematics."),
        ("Data Preprocessing", "Prepare clean data for model training."),
        ("Supervised Learning", "Learn regression and classification."),
        ("Unsupervised Learning", "Learn clustering and pattern discovery."),
        ("Machine Learning Project", "Build and evaluate a machine learning model.")
    ],
    "Artificial Intelligence": [
        ("Python Fundamentals", "Build the programming foundation."),
        ("Mathematics for AI", "Study probability, statistics and vectors."),
        ("Machine Learning Basics", "Understand training and prediction."),
        ("Neural Networks", "Learn neurons, layers and activation functions."),
        ("Artificial Intelligence Project", "Apply AI to a real-world problem.")
    ],
    "Data Analytics": [
        ("Excel Fundamentals", "Learn formulas, sorting and filtering."),
        ("SQL Fundamentals", "Retrieve useful information from databases."),
        ("Python Analytics", "Analyze datasets using Python."),
        ("Power BI", "Create interactive dashboards."),
        ("Analytics Project", "Complete an end-to-end data analysis.")
    ],
    "Web Development": [
        ("HTML", "Create the structure of web pages."),
        ("CSS", "Design attractive and responsive interfaces."),
        ("JavaScript", "Add logic and interactivity."),
        ("Frontend Development", "Build interactive frontend applications."),
        ("Full Stack Project", "Create a complete web application.")
    ]
}

# =====
# Q-LEARNING FUNCTIONS
# =====

def get_state(skill_level, goal, style, progress):
    """
    State represents the learner's current condition.
    """
    return f"{skill_level}_{goal}_{style}_{progress}"

def initialize_q_values(state, actions):
    if state not in st.session_state.q_table:
        st.session_state.q_table[state] = {
            action: 0.0 for action in actions
        }

def choose_action(state, actions, epsilon=0.20):
    """
    Epsilon-greedy action selection.
    Exploration: choose random topic.
    Exploitation: choose topic with highest Q-value.
    """
    initialize_q_values(state, actions)

    if random.random() < epsilon:
        return random.choice(actions)

    q_values = st.session_state.q_table[state]
    return max(q_values, key=q_values.get)

def update_q_table(state, action, reward, next_state, next_actions,
    alpha=0.7, gamma=0.8):
    """
    Q-learning update equation:


$$Q(s,a) = Q(s,a) + \alpha *$$

```

```

    [reward + gamma * max_a Q(next_state,next_action) - Q(s,a)]

"""
initialize_q_values(state, [action])
initialize_q_values(next_state, next_actions)

current_q = st.session_state.q_table[state][action]
next_max_q = max(st.session_state.q_table[next_state].values())

new_q = current_q + alpha * (
    reward + gamma * next_max_q - current_q
)

st.session_state.q_table[state][action] = new_q

def generate_learning_path(skill_level, goal, style, daily_hours):
    topics = LEARNING_PATHS[goal]

    progress = st.session_state.progress
    state = get_state(skill_level, goal, style, progress)

    actions = [topic[0] for topic in topics]

    selected_topics = []
    remaining_actions = actions.copy()

    for _ in range(min(5, len(topics))):
        if not remaining_actions:
            break

        action = choose_action(
            state,
            remaining_actions,
            epsilon=0.15
        )

        selected_topics.append(
            next(topic for topic in topics if topic[0] == action)
        )

        remaining_actions.remove(action)

        next_progress = min(progress + 20, 100)
        next_state = get_state(
            skill_level,
            goal,
            style,
            next_progress
        )

        reward = 10

        if style == "Projects" and "Project" in action:
            reward += 5

        if daily_hours >= 4:
            reward += 2

        update_q_table(
            state,
            action,
            reward,
            next_state,
            remaining_actions if remaining_actions else actions
        )

        state = next_state
        progress = next_progress

    return selected_topics

# =====
# SIDEBAR
# =====

with st.sidebar:
    st.markdown("## 📖 LearnPath AI")
    st.caption("AI-Powered Learning Assistant")

    st.markdown("---")
    st.markdown("### Student Preferences")

    student_name = st.text_input(
        "Student Name",
        value="Laxmi"
    )

    skill_level = st.selectbox(
        "Skill Level",
        ["Beginner", "Intermediate", "Advanced"]
    )

    learning_goal = st.selectbox(
        "Learning Goal",
        [
            "Python Programming",
            "Data Science",
            "Machine Learning",
            "Artificial Intelligence",
            "Data Analytics",
            "Web Development"
        ]
    )

```

```

learning_style = st.selectbox(
    "Learning Style",
    ["Practice", "Videos", "Reading", "Projects"]
)

daily_hours = st.slider(
    "Daily Study Hours",
    min_value=1,
    max_value=8,
    value=2
)

st.markdown("---")

generate_button = st.button(
    "🔥 Generate Learning Path",
    use_container_width=True
)

reset_button = st.button(
    "🔄 Reset Progress",
    use_container_width=True
)

if reset_button:
    st.session_state.q_table = {}
    st.session_state.recommended_path = []
    st.session_state.progress = 0
    st.session_state.reward = 0
    st.rerun()

st.markdown("---")
st.caption("Q-Learning Based Recommendation System")

# =====
# HEADER
# =====

st.markdown(
    '<div class="main-heading">Learning Dashboard</div>',
    unsafe_allow_html=True
)

st.markdown(
    '<div class="sub-heading">Track your progress and discover your personalized learning journey.</div>',
    unsafe_allow_html=True
)

# =====
# HERO
# =====

st.markdown(f"""
<div class="hero">
    <div class="badge">SMART LEARNING • Q-LEARNING RECOMMENDATION</div>
    <div class="hero-title">Hello, {student_name}! 🌟</div>
    <div class="hero-text">
        Your personalized learning path is generated using your skill level,
        learning goal, preferred learning style and available study time.
        The Q-Learning model improves recommendations using rewards.
    </div>
</div>
""", unsafe_allow_html=True)

# =====
# METRICS
# =====

c1, c2, c3, c4 = st.columns(4)

with c1:
    st.markdown(f"""
    <div class="metric-card">
        <div class="metric-label">CURRENT LEVEL</div>
        <div class="metric-value">🎯 {skill_level}</div>
    </div>
    """, unsafe_allow_html=True)

with c2:
    st.markdown(f"""
    <div class="metric-card">
        <div class="metric-label">DAILY HOURS</div>
        <div class="metric-value">🕒 {daily_hours} hrs</div>
    </div>
    """, unsafe_allow_html=True)

with c3:
    st.markdown(f"""
    <div class="metric-card">
        <div class="metric-label">LEARNING GOAL</div>
        <div class="metric-value">🎯 {learning_goal}</div>
    </div>
    """, unsafe_allow_html=True)

with c4:
    st.markdown(f"""
    <div class="metric-card">
        <div class="metric-label">PROGRESS</div>
        <div class="metric-value">📊 {st.session_state.progress}%</div>
    </div>
    """, unsafe_allow_html=True)

# =====
# FEATURE CARDS
# =====

```

```

st.markdown(
    '<div class="section-title">🌟 Why LearnPath AI?</div>',
    unsafe_allow_html=True
)

f1, f2, f3 = st.columns(3)

with f1:
    st.markdown("""
    <div class="course-card">
        <div class="course-icon">🎓</div>
        <div class="course-title">Personalized Recommendations</div>
        <div class="course-description">
            Learning topics are selected according to student preferences,
            skill level and learning goal.
        </div>
    </div>
    """, unsafe_allow_html=True)

with f2:
    st.markdown("""
    <div class="course-card">
        <div class="course-icon">📊</div>
        <div class="course-title">Progress Tracking</div>
        <div class="course-description">
            Track your learning progress and update the recommendation model
            using feedback and rewards.
        </div>
    </div>
    """, unsafe_allow_html=True)

with f3:
    st.markdown("""
    <div class="course-card">
        <div class="course-icon">🧠</div>
        <div class="course-title">Actual Q-Learning</div>
        <div class="course-description">
            The system uses state, action, reward and Q-value updates
            to select suitable learning topics.
        </div>
    </div>
    """, unsafe_allow_html=True)

# =====
# GENERATE RECOMMENDATION
# =====

if generate_button:
    st.session_state.recommended_path = generate_learning_path(
        skill_level,
        learning_goal,
        learning_style,
        daily_hours
    )

    st.session_state.reward = len(
        st.session_state.recommended_path
    ) * 10

    st.success("Your personalized learning path has been generated using Q-Learning.")

# =====
# RECOMMENDATION SECTION
# =====

if st.session_state.recommended_path:

    st.markdown(
        '<div class="section-title">🗺️ Your Recommended Learning Path</div>',
        unsafe_allow_html=True
    )

    st.write(
        f"Recommended roadmap for **{learning_goal}**:"
    )

    recommended_path = st.session_state.recommended_path
    total_topics = len(recommended_path)
    estimated_days = total_topics * 3

    m1, m2, m3 = st.columns(3)

    with m1:
        st.metric("Learning Topics", total_topics)

    with m2:
        st.metric("Estimated Days", estimated_days)

    with m3:
        st.metric("Current Reward", st.session_state.reward)

    st.markdown("### 🗺️ Roadmap Steps")

    for number, item in enumerate(recommended_path, start=1):
        topic, description = item

        st.markdown(f"""
        <div class="roadmap">
            <div class="step-number">STEP {number:02d}</div>
            <div class="step-title">{topic}</div>
            <div class="step-description">{description}</div>
        </div>
        """, unsafe_allow_html=True)

```

```
st.markdown( """ 📌 Update Your Progress """ )
```

```
new_progress = st.slider(
    "How much of your roadmap have you completed?",
    min_value=0,
    max_value=100,
    value=st.session_state.progress
)

if new_progress != st.session_state.progress:
    old_progress = st.session_state.progress
    st.session_state.progress = new_progress

    if new_progress > old_progress:
        st.session_state.reward += 10
        st.success("Progress increased. Positive reward added to the Q-Learning model.")
```

```
st.progress(st.session_state.progress / 100)
```

```
if st.session_state.progress == 0:
    st.info("Start your first topic to begin your learning journey.")
elif st.session_state.progress < 50:
    st.info("Good start! Continue learning consistently.")
elif st.session_state.progress < 100:
    st.success("Great progress! You are more than halfway there.")
else:
    st.success("🎉 Congratulations! You completed your learning roadmap.")
```

```
# =====
# RL MODEL EXPLANATION
# =====
```

```
st.markdown("### 🧠 Reinforcement Learning Model")
```

```
r1, r2, r3, r4 = st.columns(4)
```

```
with r1:
    st.markdown("""
    <div class="info-panel">
        <h4>State</h4>
        <p>
            Skill level, learning goal, learning style and current progress.
        </p>
    </div>
    """, unsafe_allow_html=True)
```

```
with r2:
    st.markdown("""
    <div class="info-panel">
        <h4>Action</h4>
        <p>
            Selecting the next suitable learning topic.
        </p>
    </div>
    """, unsafe_allow_html=True)
```

```
with r3:
    st.markdown("""
    <div class="info-panel">
        <h4>Reward</h4>
        <p>
            Positive reward is provided when the learner completes a topic
            or makes progress.
        </p>
    </div>
    """, unsafe_allow_html=True)
```

```
with r4:
    st.markdown("""
    <div class="info-panel">
        <h4>Q-Value</h4>
        <p>
            Q-values are updated to improve future topic recommendations.
        </p>
    </div>
    """, unsafe_allow_html=True)
```

```
# =====
# DOWNLOAD ROADMAP
# =====
```

```
result_df = pd.DataFrame({
    "Step": range(1, total_topics + 1),
    "Topic": [item[0] for item in recommended_path],
    "Description": [item[1] for item in recommended_path]
})
```

```
st.download_button(
    "📄 Download Learning Roadmap",
    data=result_df.to_csv(index=False),
    file_name="learning_roadmap.csv",
    mime="text/csv"
)
```

```
else:
    st.markdown("""
    <div class="info-panel">
        <h4>Ready to start?</h4>
        <p>
            Select your preferences from the sidebar and click
            <b>Generate Learning Path</b> to create your personalized
            Q-Learning based roadmap.
        </p>
    </div>
    """, unsafe_allow_html=True)
```



```
# =====
# DATASET STATUS
# =====

with st.expander("📁 Dataset Status"):
    if dataset is not None:
        st.success("Dataset loaded successfully from the project folder.")
        st.write("Rows:", dataset.shape[0])
        st.write("Columns:", dataset.shape[1])
        st.dataframe(dataset.head(10))
    elif dataset_error:
        st.error("Dataset found but could not be loaded.")
        st.write(dataset_error)
    else:
        st.info(
            "Dataset file was not found. The application is running with "
            "the built-in learning roadmap."
        )

# =====
# FOOTER
# =====

st.markdown("""
<div class="footer">
    LearnPath AI • Learning Path Recommendation using Reinforcement Learning
</div>
""", unsafe_allow_html=True)
```

Writing app.py

Obs-The server was started successfully using app.py, allowing the learning path recommendation system to run through the local web application.

```
!streamlit run app.py --server.port 8501 --server.address 0.0.0.0 &>/content/streamlit.log &
```

Obs-The local server URL was generated successfully and can be opened in a web browser to access the learning path recommendation application. LINK-<https://fmltae1-69kar6mma6dubyp2akuem8.streamlit.app/>